

DSA REVISION BIBLE

C-CAT Section B — Complete Study Sheet

Zero to Exam-Ready in 1–2 Hours

8 Topics	50+ Concepts	All Complexities	Exam Traps
Fully Covered	Key Facts Only	Best / Avg / Worst	Highlighted

Prepared for C-CAT 2024–25 • Section B: Data Structures & Algorithms

TABLE OF CONTENTS

Ch 01	Algorithm Complexity & Big-O Big-O ranking, growth order, identifying complexity from code
Ch 02	Sorting Algorithms Bubble, Selection, Insertion, Merge, Quick, Heap Sort — full properties
Ch 03	Searching Algorithms Linear Search, Binary Search — complexity & conditions
Ch 04	Stack Operations, applications, infix $\leftrightarrow$ postfix conversion & evaluation
Ch 05	Queue Types, operations, circular queue, priority queue, deque
Ch 06	Linked List Types, operations, Floyd's cycle detection, insertion order
Ch 07	Trees BST, AVL, Heap, traversals — Inorder / Preorder / Postorder
Ch 08	Graphs BFS, DFS, adjacency matrix vs list, MST — Kruskal's & Prim's
Ch BONUS	Algorithm Paradigms Greedy, Divide & Conquer, Dynamic Programming — key examples

Chapter 01: Algorithm Complexity & Big-O

1.1 Big-O Growth Order (slowest → fastest)

Big-O describes how runtime grows as input size n increases. Memorise this order cold.

Notation	Name	Example Algorithm / Operation
$O(1)$	Constant	Array index access, Hash table lookup (avg)
$O(\log n)$	Logarithmic	Binary Search, BST search (avg)
$O(n)$	Linear	Linear Search, single loop over n elements
$O(n \log n)$	Linearithmic	Merge Sort, Heap Sort, Quick Sort (avg)
$O(n^2)$	Quadratic	Bubble / Selection / Insertion Sort (worst)
$O(2^n)$	Exponential	Recursive Fibonacci, Power Set generation
$O(n!)$	Factorial	Permutation generation — all arrangements

1.2 Common Operation Complexities

Operation	Time	Notes
Array: access by index	$O(1)$	Direct jump — no traversal needed
Array: insert at beginning	$O(n)$	Must shift all elements right
Array: insert at end	$O(1)$	Just place at next available slot
Linear Search (unsorted)	$O(n)$	Must check every element
Binary Search (sorted)	$O(\log n)$	Halves search space each step
BST Search (balanced)	$O(\log n)$	Height = $\log n$ in balanced tree
BST Search (skewed)	$O(n)$	Degrades to linked list structure
Recursive Fibonacci	$O(2^n)$	Each call branches into 2 more

1.3 Identifying Complexity from Code Patterns

Code Pattern	Complexity	Example
Single loop from 0 to n	$O(n)$	for i in range(n)
Two nested loops (both size n)	$O(n^2)$	for i : for j (both n iterations)
Loop that halves each iteration	$O(\log n)$	Binary Search, while $n > 1$: $n = n/2$
Loop + inner halving	$O(n \log n)$	Merge Sort — split + merge step
Three nested loops	$O(n^3)$	Matrix multiplication
Recursion $T(n) = 2T(n/2) + n$	$O(n \log n)$	Merge Sort recurrence relation
Recursion $T(n) = T(n-1) + 1$	$O(n)$	Simple tail recursion

■ **EXAM TRAP:** $O(n \log n)$ is NOT $O(n^2)$. Merge Sort = $O(n \log n)$. These are very different — $n \log n$ is much faster.

■ EXAM TRAP: Recursive Fibonacci = $O(2^n)$, NOT $O(n)$. Each call creates 2 more branches — exponential blowup.

■ EXAM TRAP: Binary Search best case = $O(1)$ when target is at mid on first check. Average/Worst = $O(\log n)$.

Chapter 02: Sorting Algorithms

2.1 Master Complexity Table

Most exam-critical table in this entire document. Every cell has appeared in C-CAT style MCQs.

Algorithm	Best	Average	Worst	Space	Stable?	In-Place?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓ Yes	✓ Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗ No	✓ Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓ Yes	✓ Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓ Yes	✗ No
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	✗ No	✓ Yes
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗ No	✓ Yes
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	✓ Yes	✗ No

k = range of input values (Counting Sort) | Quick Sort space = recursion stack depth

2.2 Key Concepts — Stable & In-Place Defined

Term	Plain Language	Example
Stable	Equal elements KEEP their original relative order after sorting.	Raj(50), Priya(50) → stays Raj then Priya after sort
In-Place	Sorting happens within the original array. No extra array needed. Space = $O(1)$.	Bubble Sort swaps within same array — no new array created
NOT In-Place	Needs an extra temporary array. Space = $O(n)$.	Merge Sort creates a temp array to merge two halves

2.3 Algorithm Behaviour Snapshots

Bubble Sort	Compares and swaps adjacent elements repeatedly. Each pass bubbles the largest unsorted element to its correct position at the end.	Best case $O(n)$ ONLY with early-exit flag on already-sorted array. Without flag = always $O(n^2)$.
Selection Sort	Finds the minimum element and places it at correct position. Scans the FULL remaining array every pass. No early exit possible.	ALWAYS $O(n^2)$ — even on a sorted array. No optimisation exists.
Insertion Sort	Picks each element and inserts it into its correct position in the already-sorted left portion. Like sorting playing cards in hand.	Best case $O(n)$ = already sorted needs only n comparisons and 0 swaps. Best for small or nearly-sorted arrays.
Merge Sort	Divide & Conquer. Splits array in half recursively, sorts each half, then merges. ALWAYS $O(n \log n)$ — no variation.	Uses $O(n)$ extra space for merge step. Preferred for Linked Lists and external sorting.
Quick Sort	Divide & Conquer. Picks a pivot, partitions array around it, recurses on both sides. Fastest in practice due to cache efficiency.	Worst case $O(n^2)$ when pivot is always min or max (e.g. already-sorted array with first element as pivot).

Heap Sort

Builds a Max-Heap, then repeatedly extracts the maximum to sort. Always $O(n \log n)$ and in-place but NOT stable.

Slower than Quick Sort in practice due to poor cache behaviour. $O(1)$ space advantage over Merge Sort.

2.4 When to Use Which Sort

Situation	Best Choice	Reason
Need guaranteed $O(n \log n)$	Merge Sort	Quick Sort risks $O(n^2)$ worst case
Memory is very limited	Heap Sort	$O(1)$ space, always $O(n \log n)$
Stability required	Merge Sort / Insertion	Both stable algorithms
Array is nearly sorted	Insertion Sort	$O(n)$ best case kicks in
General purpose, fastest avg	Quick Sort	Cache efficient in practice
Sorting a Linked List	Merge Sort	No random access needed
Small array (< 20 elements)	Insertion Sort	Low overhead for tiny input

■ EXAM TRAP: Merge Sort is $O(n \log n)$ always — no best/worst variation. Quick Sort DOES vary (worst = $O(n^2)$).

■ EXAM TRAP: Selection Sort is ALWAYS $O(n^2)$ — even on a sorted array. There is no early exit.

■ EXAM TRAP: Quick Sort worst case = already sorted array with first or last element chosen as pivot.

■ EXAM TRAP: Heap Sort and Merge Sort both guarantee $O(n \log n)$ worst case. But Heap Sort is in-place ($O(1)$); Merge Sort is not ($O(n)$).

■ EXAM TRAP: Only Insertion Sort is BOTH stable AND in-place among the comparison sorts in this list.

■ Chapter 03: Searching Algorithms

3.1 Complexity Comparison

Algorithm	Best	Average	Worst	Prerequisite	Works On
Linear Search	$O(1)$	$O(n)$	$O(n)$	None — any array	Array or Linked List
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	Array MUST be sorted	Sorted Array only

3.2 How Binary Search Works — Step by Step

Step 1: Check the middle element of the current range.
Step 2: If target = mid → FOUND. Return index.
Step 3: If target < mid → discard right half. Search left half only.
Step 4: If target > mid → discard left half. Search right half only.
Step 5: Repeat until found or range becomes empty → NOT FOUND.

3.3 Binary Search — Simulation Example

Array: [2, 5, 8, 12, 16, 23, 38] | Target: 23

Step	Current Range	Action	Result
1	[2, 5, 8, 12, 16, 23, 38]	mid = index 3 = 12	23 > 12 → go RIGHT half
2	[16, 23, 38]	mid = index 5 = 23	23 = 23 → FOUND at index 5

■ **EXAM TRAP:** Binary Search **REQUIRES** a sorted array. On unsorted input it gives **WRONG** results — not just slow ones.

■ **EXAM TRAP:** Binary Search best case = $O(1)$ when the target is exactly at mid on the very first check.

■ Chapter 04: Stack

4.1 Core Concept

Stack = LIFO (Last In, First Out). The last element pushed is the first one popped. Think of a stack of plates — you always add and remove from the top only.

4.2 Operation Complexities

Operation	Time	Space	Description
Push	$O(1)$	$O(1)$	Add element to top of stack
Pop	$O(1)$	$O(1)$	Remove element from top of stack
Peek / Top	$O(1)$	$O(1)$	View top element without removing it
Search	$O(n)$	$O(1)$	Must traverse — no direct index access
isEmpty	$O(1)$	$O(1)$	Check if stack has zero elements
Total Stack Space	—	$O(n)$	n elements stored in the stack

4.3 Stack Applications

Application	Why Stack is Used
Infix $\rightarrow$ Postfix / Prefix conversion	Handles operator precedence using LIFO ordering
Postfix expression evaluation	Operands pushed; operator pops two and pushes result
Balancing parentheses / brackets	Push open bracket; pop and match on close bracket
Function call management	Each function call pushed onto call stack; return pops it
Undo operation (text editors)	Last action is undone first — perfect LIFO behaviour
Recursion (internally)	Each recursive call is pushed onto the system stack
DFS graph traversal	Uses stack to track which vertices to visit next
Browser back button	Previous pages stored in stack — most recent first

4.4 Expression Notations

Notation	Operator Position	Example	Used By
Infix	Between operands	$A + B$	Humans — natural math writing
Postfix (RPN)	After operands	$A B +$	Computers — no brackets needed
Prefix	Before operands	$+ A B$	Some compilers and parsers

4.5 Infix $\rightarrow$ Postfix Conversion Rules

Rule 1 — Operand (A, B, numbers): Write DIRECTLY to output. Always. No thinking needed.

Rule 2 — Opening bracket '(': PUSH onto stack. Always.

Rule 3 — Closing bracket ')': POP from stack and write to output until '(' found. Discard both brackets — never write them.

Rule 4 — Operator (+, -, *, /):

→ Stack empty: PUSH it.

→ Top has equal or higher precedence: POP top to output first, then PUSH current.

→ Top has lower precedence: just PUSH current.

End of expression: POP ALL remaining operators from stack to output.

Precedence order (highest to lowest): $^ > * / > + -$

4.6 Infix → Postfix — Worked Examples

Infix Expression	Postfix Result	Key Reason
$A + B * C$	$A B C * +$	$*$ has higher precedence than $+$ → processed before $+$
$(A + B) * C$	$A B + C *$	Brackets force $A+B$ to be evaluated first
$(A + B) * (C - D)$	$A B + C D - *$	Two bracket groups processed, then $*$ at end
$A + B + C$	$A B + C +$	Same precedence — process left to right

4.7 Postfix Evaluation Rules

Rule 1 — Operand (number): PUSH onto stack.

Rule 2 — Operator: POP two operands. Apply operator. PUSH result back onto stack.

Order matters: First popped = right operand. Second popped = left operand.

Final answer: The last remaining value on the stack.

Postfix	Trace	Result
$2\ 3\ 4\ * +$	Push 2 → Push 3 → Push 4 → $(*)$ pop 3,4 → push 12 → $(+)$ pop 2,12 → push 14	14
$5\ 1\ 2 + 4\ * + 3 -$	Push 5,1,2 → $(+)$ pop 1,2=3 → push 3 → Push 4 → $(*)$ pop 3,4=12 → $(+)$ pop 5,12=17 → Push 3 → $(-)$ = 14	14

■ **EXAM TRAP:** Operands ALWAYS go straight to output in conversion. NEVER push an operand onto the stack.

■ **EXAM TRAP:** When you see ')' — pop until '(' but do NOT write brackets in the output. Brackets disappear entirely.

■ Chapter 05: Queue

5.1 Core Concept

Queue = FIFO (First In, First Out). The first element added is the first one removed. Think of a ticket counter line — first person in line is served first.

5.2 Operation Complexities

Operation	Simple Queue	Circular Queue	Notes
Enqueue (add)	$O(1)$	$O(1)$	Add element at the rear
Dequeue (remove)	$O(1)$	$O(1)$	Remove element from the front
Peek / Front	$O(1)$	$O(1)$	View front without removing
Search	$O(n)$	$O(n)$	Must traverse — no direct access
isEmpty	$O(1)$	$O(1)$	Check if front == rear

5.3 Types of Queue

Type	Key Property & Use Case
Simple Queue	FIFO. Fixed front and rear. Rear cannot wrap around. Wasted space problem exists. Use: basic task scheduling.
Circular Queue	Rear wraps back to front when end is reached. REUSES empty slots. Solves the wasted-space problem. Use: CPU scheduling, buffering.
Priority Queue	Element with HIGHEST priority dequeues FIRST. NOT purely FIFO — order depends on priority. Use: Dijkstra's, OS scheduling.
Deque (Double-ended)	Insert AND delete from BOTH front and rear ends. Use: sliding window problems, palindrome checking.
Input Restricted Deque	Insert at rear only. Delete from both ends. Specialised scheduling use case.
Output Restricted Deque	Delete from front only. Insert from both ends. Specialised buffering use case.

5.4 Applications of Queue

Application	Why Queue is Used
BFS graph traversal	Processes nodes level by level — naturally FIFO order
CPU process scheduling	Processes are served in the order they arrive
Printer spooling	Print jobs are processed in the order received
Keyboard buffer	Keystrokes are processed in the order typed
Call center systems	First caller in queue is the first one served

■ **EXAM TRAP:** Stack = LIFO. Queue = FIFO. C-CAT describes behaviour and asks which structure — don't mix up under pressure.

■ EXAM TRAP: Circular Queue does NOT increase array size. It REUSES empty slots at the front. Same fixed size, smarter use.

■ EXAM TRAP: Priority Queue is NOT a simple FIFO queue. Dequeue order depends on priority, NOT arrival time.

Chapter 06: Linked List

6.1 What is a Node?

Singly LL Node: [DATA | NEXT →] — one pointer to the next node

Doubly LL Node: [← PREV | DATA | NEXT →] — two pointers

Last node's NEXT: Points to NULL (non-circular) or HEAD (circular)

Head pointer: Always points to the FIRST node in the list

6.2 Types of Linked List

Type	Direction	Last Node Points To	Memory Per Node
Singly Linked List	One-way →	NULL	1 pointer per node
Doubly Linked List	Both ways ←→	NULL	2 pointers per node
Circular Singly	One-way →	HEAD (1st node)	1 pointer per node
Circular Doubly	Both ways ←→	HEAD (1st node)	2 pointers per node

6.3 Operation Complexities

Operation	Singly LL	Doubly LL	Array (comparison)
Access by index	$O(n)$	$O(n)$	$O(1)$
Search	$O(n)$	$O(n)$	$O(n)$
Insert at beginning	$O(1)$	$O(1)$	$O(n)$
Insert at end	$O(n)^*$	$O(n)^*$	$O(1)$
Insert at middle	$O(n)$	$O(n)$	$O(n)$
Delete at beginning	$O(1)$	$O(1)$	$O(n)$
Delete at end	$O(n)$	$O(n)$	$O(1)$
Delete (given node ptr only)	$O(n)$	$O(1)$	—

* $O(1)$ if a separate tail pointer is maintained

6.4 Singly vs Doubly — Key Trade-offs

Property	Singly LL	Doubly LL
Pointers per node	1 (next only)	2 (next + prev)
Memory per node	Less	More
Reverse traversal	Not possible	Possible
Delete (given only node ptr)	$O(n)$ — need previous node	$O(1)$ — use prev pointer
Implementation complexity	Simpler	More complex

6.5 Linked List vs Array — When to Use Which

Situation	Choose	Reason
Frequent insert/delete at beginning	Linked List	$O(1)$ vs $O(n)$ for array
Frequent access by index	Array	$O(1)$ vs $O(n)$ for linked list
Size changes dynamically	Linked List	No reallocation needed
Memory is fixed and known	Array	Less overhead per element
Random access needed frequently	Array	No direct index in linked list
Sorting (Merge Sort preferred)	Linked List	Merge Sort needs no random access

6.6 Floyd's Cycle Detection Algorithm

Purpose: Detect if a linked list contains a loop/cycle.

Two pointers: SLOW moves 1 step at a time. FAST moves 2 steps at a time.

Cycle exists: If SLOW and FAST pointers meet at the same node.

No cycle: If FAST pointer reaches NULL.

Time complexity: $O(n)$ | **Space complexity:** $O(1)$ — no extra array needed

6.7 Node Insertion — Step Order Matters

Task: Insert node 25 between 20 and 30 in: HEAD $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ NULL

Step 1 (DO THIS FIRST): Set 25's NEXT = 30 (point new node to the next node FIRST)

Step 2 (DO THIS SECOND): Set 20's NEXT = 25 (point previous node to the new node)

If you do Step 2 first, you permanently lose the reference to 30 — the list breaks!

■ **EXAM TRAP:** Accessing an element in Linked List = $O(n)$. No index jumping like arrays. Must traverse from HEAD every time.

■ **EXAM TRAP:** Insert at BEGINNING = $O(1)$. Insert at END = $O(n)$. Many students assume both are $O(1)$ — they are not.

■ **EXAM TRAP:** Circular Linked List: the last node points to the FIRST node, NOT to NULL. Don't write NULL by habit.

■ **EXAM TRAP:** Floyd's needs exactly TWO pointers — slow and fast. Not one, not three.

■ Chapter 07: Trees

7.1 Core Terminology

Term	Plain Language Definition
Root	Top node of the tree. Has no parent. There is exactly one root.
Parent	A node that has at least one child node directly below it.
Child	A node that comes directly below a parent node.
Leaf	A node with NO children. The bottom-most nodes of the tree.
Height	Longest path (in edges) from root to any leaf. Root alone = height 0.
Depth	Distance (in edges) of a node from the root. Root depth = 0.
Degree	Number of children a node has.
Subtree	A node and all of its descendants taken together.

7.2 Types of Trees

Type	Key Rule	C-CAT Focus
Binary Tree	Each node has AT MOST 2 children.	Foundation for BST and Heap
Full Binary Tree	Every node has EITHER 0 or 2 children. No node has exactly 1 child.	Structure-based questions
Complete Binary Tree	All levels fully filled except possibly the last; last level filled left to right.	Used as the basis for Heap
Binary Search Tree (BST)	Left subtree < Node value < Right subtree. This rule holds for EVERY node.	Search, Insert, Delete ops
AVL Tree	BST + self-balancing. Balance factor of every node must be -1, 0, or +1.	Guaranteed $O(\log n)$ always
Heap	Complete binary tree. Max-Heap: parent $\geq$ children. Min-Heap: parent $\leq$ children.	Heap Sort, Priority Queue

7.3 BST Complexities

Operation	Average (Balanced)	Worst (Skewed)
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$
Inorder Traversal	$O(n)$	$O(n)$

Skewed BST: All nodes go to one side — looks exactly like a linked list. Example: inserting 10, 20, 30, 40 in order creates a fully right-skewed BST. Search on a skewed BST = $O(n)$, NOT $O(\log n)$.

7.4 Tree Traversals — The Big Three

Traversal	Order	Memory Trick	Output (Root=10, L=6(4,8), R=15(_,20))
Inorder	Left → Root → Right	L Ro R	4, 6, 8, 10, 15, 20
Preorder	Root → Left → Right	Ro L R	10, 6, 4, 8, 15, 20
Postorder	Left → Right → Root	L R Ro	4, 8, 6, 20, 15, 10
Level Order	Level by level, left to right	Use a Queue	10, 6, 15, 4, 8, 20

7.5 AVL Tree

Balance Factor = Height(Left Subtree) - Height(Right Subtree)

Allowed values: -1, 0, or +1 ONLY. Any other value triggers a rotation to rebalance.

Rotation types: LL Rotation, RR Rotation, LR Rotation, RL Rotation

Guarantee: Search, Insert, Delete all = $O(\log n)$ ALWAYS — never degrades to $O(n)$.

Key difference from BST: Plain BST can degrade to $O(n)$ when skewed. AVL Tree cannot.

7.6 Heap

Operation	Time	Notes
Get Max (Max-Heap)	$O(1)$	Root is always the maximum — no traversal needed
Get Min (Min-Heap)	$O(1)$	Root is always the minimum — no traversal needed
Insert	$O(\log n)$	Insert at end, then bubble UP (heapify up)
Delete Max / Min	$O(\log n)$	Replace root with last node, then bubble DOWN
Build Heap from array	$O(n)$	NOT $O(n \log n)$ — this surprises most students

Max-Heap rule: Every parent is $\geq$ its children. Root = largest element in the heap.

Min-Heap rule: Every parent is $\leq$ its children. Root = smallest element in the heap.

IMPORTANT: Heap is NOT a BST. The left $<$ root $<$ right rule does NOT apply in a Heap.

■ **EXAM TRAP:** Inorder traversal of a BST ALWAYS gives sorted ascending output. This is a direct C-CAT exam fact.

■ **EXAM TRAP:** AVL Tree IS a BST — it has ALL BST properties PLUS self-balancing. It is NOT a separate structure.

■ **EXAM TRAP:** Heap is NOT a BST. No left $<$ root $<$ right rule applies. Only the parent-child relationship matters.

■ **EXAM TRAP:** BST worst case = $O(n)$ when skewed. AVL Tree worst case = $O(\log n)$ ALWAYS. C-CAT tests this difference.

■ **EXAM TRAP:** Building a Heap from an array = $O(n)$, NOT $O(n \log n)$. This is a frequently tested surprise.

■ Chapter 08: Graphs

8.1 Core Terminology

Term	Definition
Vertex (Node)	A point or entity in the graph.
Edge	A connection/link between two vertices.
Degree	Number of edges directly connected to a vertex.
Path	A sequence of vertices connected by edges.
Cycle	A path that starts and ends at the same vertex.
Connected Graph	Every vertex can be reached from every other vertex.
Disconnected	Some vertices cannot be reached from others.
DAG	Directed Acyclic Graph — directed edges with no cycles.

8.2 Types of Graphs

Type	Key Property
Undirected	Edges have NO direction. A-B means travel is possible both ways.
Directed (Digraph)	Edges have a direction. A→B means you can go A to B only, not B to A.
Weighted	Edges have an associated cost or weight (e.g. distance, time, money).
Unweighted	Edges have no cost — simply connected or not connected.
Cyclic	Contains at least one cycle (a path that loops back to the start).
Acyclic	No cycles exist. Trees are a special case of connected acyclic graphs.

8.3 Graph Representation — Matrix vs List

Property	Adjacency Matrix	Adjacency List
Structure	2D array $V \times V$. $matrix[i][j]=1$ if edge between i and j .	Each vertex stores a list of its neighbours.
Space	$O(V^2)$	$O(V + E)$
Check edge exists	$O(1)$	$O(\text{degree of vertex})$
Find all neighbours	$O(V)$	$O(\text{degree of vertex})$
Best for	Dense graphs (many edges)	Sparse graphs (few edges)
Memory efficiency	Poor for sparse graphs	Efficient for sparse graphs

8.4 BFS vs DFS — Core Comparison

Property	BFS (Breadth First)	DFS (Depth First)
Data structure used	Queue	Stack (or recursion)

Property	BFS (Breadth First)	DFS (Depth First)
Traversal style	Level by level	Branch by branch (go deep first)
Finds shortest path	YES (unweighted graphs only)	NO
Memory usage	More (stores all neighbours)	Less
Time complexity	$O(V + E)$	$O(V + E)$
Space complexity	$O(V)$	$O(V)$
Best for	Shortest path, nearby nodes	Cycle detection, maze solving, topological sort
Internal mechanism	Enqueue neighbours, dequeue to visit	Push to stack, pop to visit (or recurse)

8.5 Shortest Path Algorithms

Algorithm	Graph Type	Negative Weights?	Time Complexity
BFS	Unweighted only	N/A	$O(V + E)$
Dijkstra's	Weighted, no negative edges	No	$O(E \log V)$
Bellman-Ford	Weighted, any edges	Yes	$O(VE)$
Floyd-Warshall	All pairs shortest path	Yes	$O(V^3)$

8.6 Minimum Spanning Tree (MST)

MST: A subset of edges connecting ALL vertices with MINIMUM total weight and NO cycles.
Number of edges in any MST: Always $V-1$ (V = number of vertices)

Algorithm	Approach	How It Works Time Best For
Kruskal's	Greedy	Sort ALL edges by weight. Add the smallest edge that does NOT form a cycle. Repeat until $V-1$ edges added. $O(E \log E)$ Sparse graphs
Prim's	Greedy	Start from one vertex. Always add the cheapest edge connecting the current tree to a NEW vertex not yet included. $O(E \log V)$ Dense graphs

■ **EXAM TRAP:** BFS uses a QUEUE. DFS uses a STACK. C-CAT regularly swaps these in wrong options — memorise this cold.

■ **EXAM TRAP:** BFS finds shortest path ONLY in UNWEIGHTED graphs. For weighted graphs, use Dijkstra's algorithm.

■ **EXAM TRAP:** Adjacency Matrix space = $O(V^2)$. Adjacency List space = $O(V+E)$. List is far better for sparse graphs.

■ **EXAM TRAP:** Both BFS and DFS have the SAME time complexity $O(V+E)$. Neither is inherently faster than the other.

■ **EXAM TRAP:** Kruskal's AND Prim's are both GREEDY algorithms. They are NEVER Divide & Conquer or Dynamic Programming.

■ Chapter BONUS: Algorithm Paradigms

B.1 The Three Main Paradigms

Paradigm	Core Idea + Key Examples
Greedy	Make the LOCALLY OPTIMAL choice at each step. Never reconsider a past choice. Fast but not always globally optimal. Examples: Kruskal's MST, Prim's MST, Dijkstra's Shortest Path, Activity Selection, Huffman Coding
Divide & Conquer	Split problem into smaller subproblems of the SAME TYPE . Solve each recursively. COMBINE results. Examples: Merge Sort, Quick Sort, Binary Search, Strassen's Matrix Multiplication
Dynamic Programming	Break into OVERLAPPING subproblems. Store results (memoization / tabulation) to avoid recomputation. Examples: Fibonacci (optimized), Longest Common Subsequence, 0/1 Knapsack, Floyd-Warshall

B.2 Divide & Conquer — Three Distinct Phases

Phase 1 — DIVIDE: Split the original problem into two or more smaller subproblems of the same type.

Phase 2 — CONQUER: Solve each subproblem recursively. Base case: solve directly when small enough.

Phase 3 — COMBINE: Merge/combine the solutions of subproblems to get the final answer.

Conquer = solving the subproblems. Combine = merging their results. These are two separate phases.

B.3 Quick Identification Guide

If question mentions...	The paradigm is...
'locally optimal', 'greedy choice', Kruskal's, Prim's, Dijkstra's, Huffman Coding	Greedy
'split into halves', 'divide and solve', Merge Sort, Quick Sort, Binary Search	Divide & Conquer
'overlapping subproblems', 'memoization', Fibonacci, Knapsack, LCS	Dynamic Programming
'try all possibilities', 'backtrack', N-Queens, Sudoku solver	Backtracking
'explore all neighbours', BFS, level-order traversal	Queue-based traversal
'go deep first', DFS, cycle detection in graph	Stack-based traversal

■ **EXAM TRAP:** Dijkstra's = Greedy. NOT Dynamic Programming, even though it stores distances. It always picks the smallest distance vertex next — that's the greedy choice.

■ **EXAM TRAP:** Merge Sort and Quick Sort = Divide & Conquer. NOT Greedy. They split problems, recurse, and combine.

■ **EXAM TRAP:** 'Conquer' phase ≠ 'Combine' phase. Conquer = solving subproblems. Combine = merging results. C-CAT tests this distinction.

B.4 Master Summary — All Complexities at a Glance

Topic / Algorithm	Best	Average	Worst	Space	Stable	In-Place
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	—	—
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	—	—
BST Search (balanced)	—	$O(\log n)$	$O(n)$ skewed	$O(n)$	—	—
AVL Search	—	$O(\log n)$	$O(\log n)$	$O(n)$	—	—
Heap Insert/Delete	—	$O(\log n)$	$O(\log n)$	$O(n)$	—	—
BFS / DFS	—	$O(V+E)$	$O(V+E)$	$O(V)$	—	—
Dijkstra's	—	$O(E \log V)$	$O(E \log V)$	$O(V)$	—	—
Kruskal's MST	—	$O(E \log E)$	$O(E \log E)$	$O(V)$	—	—
Stack Push / Pop	$O(1)$	$O(1)$	$O(1)$	$O(1)$	—	—
Queue Enq / Deq	$O(1)$	$O(1)$	$O(1)$	$O(1)$	—	—